

Module 04

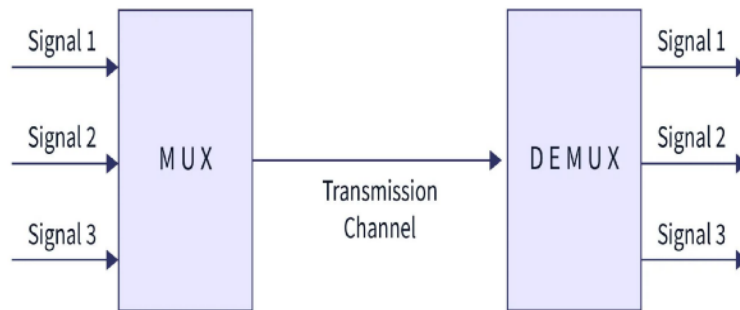
Q1 What is Transport Layer? Working of Transport Layer & Services Provided by the Transport Layer

1. **Definition of Transport Layer:**
 - The transport layer is the **4th layer** of the OSI model and is responsible for providing communication services directly to the application processes running on different hosts in a network.
2. **Role in Communication:**
 - It establishes **logical communication** between application processes across different hosts, enabling them to exchange data, even though they are not physically connected.
3. **Protocols at the Transport Layer:**
 - The transport layer uses protocols like **TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)**, each providing different communication services.
 - **TCP** is connection-oriented, reliable, and ensures data integrity, while **UDP** is connectionless and faster but less reliable.
4. **Segmentation and Reassembly:**
 - At the sender's side, the transport layer takes data from the application layer, **segments** it into smaller units, adds **header information** (e.g., port numbers), and passes it to the network layer.
 - On the receiver's side, the transport layer **reassembles** the segments and forwards them to the appropriate application process.
5. **End-to-End Communication:**
 - The transport layer facilitates **end-to-end communication** between two hosts, ensuring that data sent from one application is correctly received and processed by the destination application.
6. **Services Provided by the Transport Layer:**
 - **Multiplexing/Demultiplexing:** The transport layer allows multiple applications on the sender and receiver to share a single network channel by using **port numbers** for differentiation.
 - **Reliable Delivery:** Ensures data is delivered correctly without loss or corruption, typically through protocols like TCP that manage retransmissions and acknowledgments.
7. **Flow Control:**
 - It ensures that the sender does not overwhelm the receiver by controlling the rate of data transmission. This is especially important in protocols like TCP, which includes mechanisms to prevent congestion.
8. **Error Control:**
 - The transport layer provides **error detection** and **correction** through techniques like checksums and retransmissions, ensuring the accuracy and integrity of the data.
9. **Connection Establishment and Termination:**
 - The transport layer is responsible for establishing, maintaining, and terminating communication sessions between applications. This is achieved using protocols like TCP that use a **three-way handshake** for connection setup.
10. **Duplication Control:**
 - To prevent the arrival of duplicate data packets, the transport layer uses **sequence numbers**. This ensures that the data is delivered in order and that no duplicates reach the destination application.

Q2 Multiplexing and De-multiplexing in Computer Networks

1. **Definition of Multiplexing:**
 - **Multiplexing** is the process of combining multiple data streams from different applications or sources into a single stream for transmission over a network. This is done by adding source and destination port numbers to the data's header.
2. **Working of Multiplexing:**
 - The transport layer in protocols like **TCP** and **UDP** uses multiplexing to combine data from various application processes (using different port numbers) into one data packet. This reduces the need for multiple transmission channels and optimizes network usage.
3. **Definition of De-multiplexing:**
 - **De-multiplexing** is the reverse process where a single incoming data stream is split into multiple streams, each corresponding to the appropriate application process at the receiver's end, based on the port numbers in the header.
4. **Working of De-multiplexing:**
 - Upon receiving the multiplexed data packet, the transport layer **examines the destination port number** and directs the data to the appropriate application process. This ensures that each process receives the correct data stream.
5. **Advantages of Multiplexing:**
 - **Efficiency & Reduced Costs** : Multiplexing reduces the number of transmission channels required by combining multiple data streams into a single channel.
 - **Simplified Network Management**: By reducing the number of connections, multiplexing simplifies network management and improves system performance.
6. **Disadvantages of Multiplexing:**
 - **Complexity**: Implementing multiplexing requires sophisticated algorithms and handling of control signals.
 - **Hardware Overhead**: Multiplexers add extra hardware complexity for controlling the switching process.
 - **Synchronization Issues**: In some cases, synchronization between multiplexed data streams can lead to delays.
7. **Applications of Multiplexing:**
 - **Communication Systems**: Multiplexers are used in telecommunication systems to transmit multiple data signals (audio, video, etc.) over a single channel, optimizing bandwidth usage.
 - **Computer Networks**: In networking, multiplexing is used to send data from multiple applications (e.g., HTTP, FTP) over the same network connection.
8. **Advantages of De-multiplexing:**
 - **Signal Separation**: De-multiplexing allows the separation of a combined signal into individual application data streams, ensuring correct delivery.
 - **Increased Efficiency**: By splitting a multiplexed signal, de-multiplexing helps increase the efficiency of communication systems.
9. **Disadvantages of De-multiplexing:**
 - **Synchronization Delays**: De-multiplexing requires precise timing, which can introduce delays if synchronization between streams is not handled properly.
 - **Complexity in Handling Multiple Streams**: The receiver must manage and identify multiple data streams, which can lead to higher computational overhead.
10. **Difference Between Multiplexing and De-multiplexing:**
 - **Multiplexing**: Combines multiple data streams into one stream for transmission (many-to-one).
 - **De-multiplexing**: Separates a single incoming data stream into multiple output streams (one-to-many).
 - **Application in Networks**: Multiplexing occurs at the sender's side (often using **Time Division Multiplexing**), while de-multiplexing happens at the receiver's side.
 - **Functionality**: Multiplexing improves bandwidth usage, while de-multiplexing ensures data reaches the correct destination process.

Multiplexer and Demultiplexer



Q3 UDP Protocol (User Datagram Protocol)

1. Definition and Development:

- UDP (User Datagram Protocol) is a **connectionless** transport layer protocol, developed by **David P. Reed** in 1980. It is defined in **RFC 768** and is part of the **TCP/IP protocol suite**, used for transmitting data over the Internet.

2. Working Mechanism:

- UDP allows **applications** to send messages in the form of **datagrams** over the IP network. It encapsulates data into packets, adds its own **header information**, and then passes these packets to the IP layer for delivery to the destination.

3. Key Features of UDP:

- Unlike **TCP**, UDP is **connectionless**, meaning it does not establish a connection before sending data. It doesn't guarantee the **delivery**, **reliability**, or **order** of the data, making it suitable for applications requiring high performance, such as **live streaming**, **gaming**, or **video conferencing**.

4. Use Cases of UDP:

- UDP is preferred in scenarios where **speed** and **low latency** are more important than guaranteed delivery, such as in real-time applications (e.g., **voice calls**, **online gaming**, or **video chats**), where the occasional loss of data packets is acceptable.

5. Differences Between UDP and TCP:

- **TCP** is a **connection-oriented** protocol, ensuring reliable delivery of data, while **UDP** is **connectionless**, offering faster communication but without reliability guarantees.
- UDP has **less overhead** than TCP because it does not require handshakes or error-checking for each packet, making it more efficient for real-time applications.
- **TCP** provides **host-to-host communication**, while **UDP** enables **process-to-process communication**, using **port numbers** to differentiate multiple applications running on a single host.

Q4 UDP Header Format

1. Source Port Number:

- The **Source Port Number** is a 16-bit field that specifies the **port** from which the data packet is being sent. It helps the receiving system know which application process is sending the data.

2. Destination Port Number:

- The **Destination Port Number** is also a 16-bit field, used to identify the **port** on the destination machine that will receive the data. It ensures that the data reaches the correct application or service.

3. Length:

- The **Length** field is 16 bits and indicates the total size of the UDP packet, including both the header and the data. The minimum value is 8 bytes (the size of the header), and the total length of the packet varies depending on the size of the data.

4. Checksum:

- The **Checksum** field is a 16-bit value used for **error checking**. It ensures that the data has not been corrupted during transmission. Although this field is optional, it is recommended to use it for data integrity verification. If not used, the checksum field is set to zero.

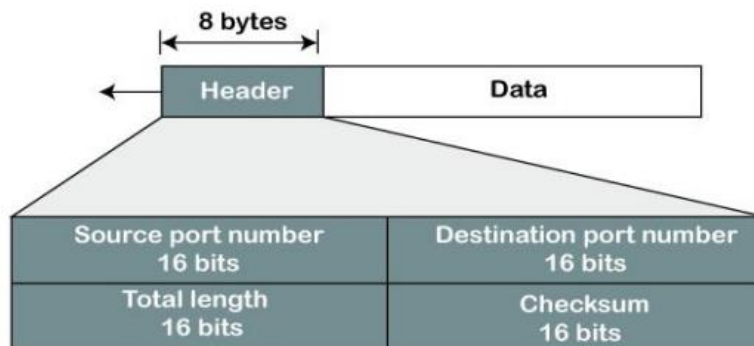
5. Key Features of UDP:

- **Acknowledgement Not Required:** UDP is suitable for applications where acknowledging each data packet is not necessary.
- **Efficient for One-Way Communication:** It is ideal for data flowing in one direction, such as in **streaming** applications.
- **Simplicity:** UDP is simple, lightweight, and efficient for **query-based** communications where speed is prioritized.
- **Stateless Protocol:** It does not maintain any session or connection information, making it stateless.
- **Unreliable Delivery:** UDP does not ensure **ordered delivery** or provide **congestion control**, which makes it faster but less reliable than TCP.

Applications of UDP

- **Domain Name Services (DNS)**
- **Simple Network Management Protocol (SNMP)**
- **Trivial File Transfer Protocol (TFTP)**
- **Routing Information Protocol (RIP)**
- **Kerberos (Authentication Protocol)**

UDP Header Format



Q5 Principle of Reliable Data Transfer Protocol

1. Definition and Purpose:

- Reliable data transfer protocols, such as **TCP (Transmission Control Protocol)**, ensure the **secure and reliable** transfer of data between application processes, even if the underlying network layer (such as IP) is unreliable.

2. Data Segmentation and Header:

- Data is divided into smaller **segments** at the sender's side. Each segment is **encapsulated** with a header, which contains necessary information for routing, sequencing, and error detection.

3. Error Handling:

- The protocol ensures that **no data is lost** or **corrupted** during transmission. This is achieved by mechanisms like **acknowledgments** and **retransmissions** in case of lost or corrupted packets.

4. Sequence Control:

- Segments are sent in sequence, and the **receiver** reassembles the data in the correct order. This prevents out-of-order delivery and ensures that data reaches the application layer in the same sequence it was sent.

5. End-to-End Service:

- **TCP** provides an **end-to-end reliable transfer** over an **unreliable network layer**. This ensures that even if the network layer experiences packet loss or corruption, the data is reliably delivered to the destination application.

Transport Layer	Network Layer
The transport layer is the fourth layer of the OSI model.	The network layer is the third layer of the OSI model.
The transport layer mainly deals with logical communication between the processes running on different hosts.	The network mainly deals with logical communication between the hosts present on the same or different network.
The transport layer focuses on the process to process the delivery of data.	The network layer provides communication between hosts of different networks.
The transport layer receives the data from the upper layer and converts it into smaller parts known as segments .	The network layer divides the data received from the transport layer in the form of packets .
The transport layer performs the port addressing i.e. the addition of a port number to the header of the data.	The network layer adds the logical address i.e. IP address (Internet Protocol address) if the packet crosses the network boundary.
The transport layer maintains the order of data.	The network layer does not focus on maintaining the order of the data packets
The transport layer deals with the process-to-process communication or port-to-port communication.	The network layer deals with host-to-host communication.
The various protocols used in the transport layer are TCP (Transmission Control Protocol), UDP (User Datagram Protocol), etc.	The various protocols used in the network layer are IPv4 , IPv6 , ICMP (Internet Control Message Protocol), etc.
The various devices used in the transport are Segments, Load Balancers, etc.	The various devices used in the network layer are Routers, Brouters, etc.

S.NO	Go-Back-N Protocol	Selective Repeat Protocol
1.	In Go-Back-N Protocol, if the sent frame are find suspected then all the frames are re-transmitted from the lost packet to the last packet transmitted.	In selective Repeat protocol, only those frames are re-transmitted which are found suspected.
2.	Sender window size of Go-Back-N Protocol is N.	Sender window size of selective Repeat protocol is also N.
3.	Receiver window size of Go-Back-N Protocol is 1.	Receiver window size of selective Repeat protocol is N.
4.	Go-Back-N Protocol is less complex.	Selective Repeat protocol is more complex.
5.	In Go-Back-N Protocol, neither sender nor at receiver need sorting.	In selective Repeat protocol, receiver side needs sorting to sort the frames.
6.	In Go-Back-N Protocol, type of Acknowledgement is cumulative.	In selective Repeat protocol, type of Acknowledgement is individual.
7.	In Go-Back-N Protocol, Out-of-Order packets are NOT Accepted (discarded) and the entire window is re-transmitted.	In selective Repeat protocol, Out-of-Order packets are Accepted.
8.	In Go-Back-N Protocol, if Receives a corrupt packet, then also, the entire window is re-transmitted.	In selective Repeat protocol, if Receives a corrupt packet, it immediately sends a negative acknowledgement and hence only the selective packet is retransmitted.
9.	Efficiency of Go-Back-N Protocol is	Efficiency of selective Repeat protocol is also

Q6 What is Transmission Control Protocol (TCP)?

1. **Connection-Oriented Protocol:**
 - **TCP (Transmission Control Protocol)** is a **connection-oriented** transport layer protocol, meaning it requires the establishment of a connection between the sender and receiver before any data transmission can occur.
2. **Reliable Data Transfer:**
 - TCP ensures reliable data transfer by dividing data into **packets**, numbering them, and sending them to the destination. It guarantees that the data is delivered correctly and in order.
3. **Data Reassembly:**
 - Upon receiving the packets, TCP at the destination **reassembles** the data into its original form and passes it to the application layer for further processing.
4. **End-to-End Communication:**
 - TCP operates on top of the **IP (Internet Protocol)**, providing end-to-end communication by managing the reliable delivery of packets from the source to the destination, ensuring no data loss or corruption.
5. **Connection Management:**
 - The connection remains **active** for the entire duration of the communication and is **terminated** once the data transfer is complete, ensuring that both sides are synchronized.

Working of TCP

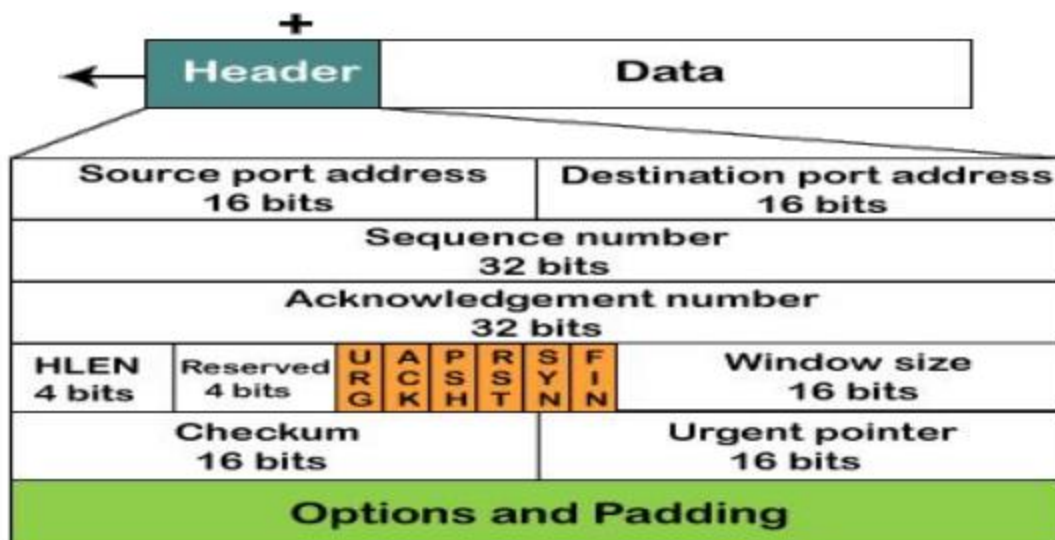
1. **Three-Way Handshake:**
 - TCP establishes a connection through a three-step process:
 - The **client** sends a segment with its sequence number.
 - The **server** responds with its sequence number and an acknowledgment number (client's sequence number +1).
 - The **client** acknowledges the server's response, establishing the connection.

Q7 Need for Transmission Control Protocol (TCP)

1. **Layered Network Architecture:**
 - In the **TCP/IP model**, the communication tasks are divided across five layers: **Application, Transport, Network, Data Link, and Physical**. The **Transport Layer** plays a crucial role in ensuring **end-to-end communication** between application processes on different systems.
2. **Multiple Application Support:**
 - TCP creates **65,000 ports**, allowing **multiple applications** to be accessed simultaneously on the same machine. This makes it highly suitable for modern networking environments with diverse services and applications.
3. **Data Segmentation and Transmission:**
 - TCP **segments** the data received from the application layer into smaller, manageable packets. These packets are then transmitted to the network layer for delivery to the destination, ensuring efficient and reliable data transfer.
4. **Reliable Data Transfer:**
 - TCP ensures **reliable data transfer** by using mechanisms such as **error checking, flow control, and sequencing** to ensure that data is delivered accurately and in order, even in the presence of network disruptions.
5. **Connection Establishment:**
 - The connection is established through a **three-way handshake**, where the client and server exchange sequence numbers and acknowledgments to confirm the connection's reliability before data transmission begins.

Q8 TCP Header Format

1. **Source Port (16 bits):**
 - This field contains the **source port address** of the application sending the data.
2. **Destination Port (16 bits):**
 - This field contains the **destination port address** where the data should be received.
3. **Sequence Number:**
 - The **sequence number** identifies the order of data bytes in the connection session, ensuring proper reassembly of packets.
4. **Acknowledgment Number:**
 - When the **ACK flag** is set, this field contains the **next sequence number** expected by the receiver, confirming the receipt of data.
5. **Header Length (HLEN):**
 - This field indicates the length of the header in **4-byte words**, with the size typically ranging between **20-60 bytes**.
6. **Reserved (4 bits):**
 - A reserved field for future use, typically set to **0** by default.
7. **Flags (Control Bits):**
 - **URG**: Indicates that the data is urgent.
 - **ACK**: Acknowledges receipt of data.
 - **PSH**: Requests immediate delivery to the application layer.
 - **RST**: Resets the connection.
 - **SYN**: Initiates connection establishment.
 - **FIN**: Ends the connection.
8. **Window Size (16 bits):**
 - Specifies the **buffer size** at the receiver's end, which helps control the flow of data and prevent congestion.
9. **Checksum (16 bits):**
 - A **mandatory field** used for error detection to verify data integrity during transmission.
10. **Urgent Pointer:**
 - Points to the **urgent data byte** when the **URG flag** is set, indicating data that should be prioritized.
11. **Options (32 bits):**
 - Provides additional settings or configurations for the connection. If the options field is less than 32 bits, **padding** is used to fill the remaining space.



Q9 Round Trip Time (RTT)

RTT (Round Trip Time) is the time taken for a network packet to travel from the sender to the receiver and then return to the sender. It's often referred to as **round-trip delay** and is a key metric in network performance assessment.

- **Unit of Measurement:** RTT is measured in **milliseconds (ms)** and is used to evaluate the health of a network. Lower RTT values indicate faster communication, while higher values suggest potential delays.
- **Significance:** RTT helps determine the speed of a network connection and can indicate issues like network congestion, slow transmission speeds, or packet loss.

Key Terminology

1. **RTT (Round Trip Time):** The time it takes for a signal or data packet to travel from the source to the destination and back to the source. It includes delays such as **propagation**, **transmission**, and **processing delays**.
2. **Latency:** Often used interchangeably with RTT, latency refers to the time delay in network communication.
3. **Network Performance:** The efficiency and reliability of a network in terms of speed, reliability, and capacity.
4. **Packet Loss:** Occurs when data packets fail to reach their destination, negatively affecting network performance.
5. **Congestion:** The state where the demand for network resources exceeds the capacity, leading to slow performance and potential packet loss.
6. **Smoothing Factor (α):** A constant used in the **Expected RTT** formula to smooth the impact of fluctuations in RTT. It balances the influence of the previous RTT measurement and the new sample RTT.

Formula to Calculate Expected RTT

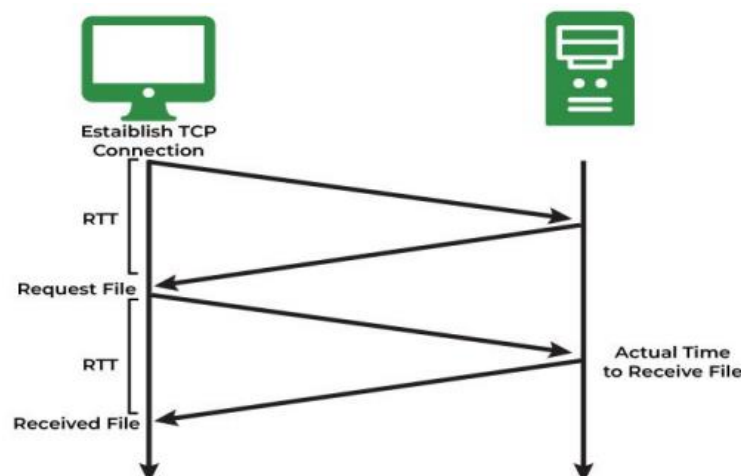
The **Expected RTT** can be calculated using the formula:

$\text{Expected RTT} = (1 - \alpha) \times \text{Previous RTT} + \alpha \times \text{Sample RTT}$

$\text{Expected RTT} = (1 - \alpha) \times \text{Previous RTT} + \alpha \times \text{Sample RTT}$

Where:

- **Previous RTT:** The RTT from the previous measurement.
- **Sample RTT:** The most recent RTT measurement.
- **α (alpha):** The **smoothing factor** (between 0 and 1) that controls the influence of the current RTT versus the previous one.



CONCEPTS – MAIN POINTS GIVEN BELOW [COOK YOUR OWN ANSWER USING THIS POINTS]

TCP Transport Layer and Flow Control

1. **Producer (Application Layer):** The application layer generates data that needs to be sent across the network.
2. **Consumer (Transport Layer):** The transport layer receives data from the application layer, divides it into packets, and forwards it to the network layer for transmission.
3. **Flow Control Mechanisms:** The transport layer uses flow control to ensure the sender doesn't overwhelm the receiver, preventing packet loss or excessive delays. Techniques include:
 - **Window-based flow control:** Adjusts the rate of transmission based on the receiver's ability to process data.
 - **Producer acknowledgment:** In messaging systems, the broker sends acknowledgments to the producer to indicate readiness to handle more data.

TCP Connection Management

- **Connection Establishment (Three-Way Handshake):**
 1. **Client** sends a SYN (synchronize) segment.
 2. **Server** responds with SYN-ACK (synchronize-acknowledge).
 3. **Client** sends ACK (acknowledge) to complete the handshake. This establishes a reliable connection between the two hosts.
- **Call Collision:** If two hosts try to establish a connection at the same time on the same port, only one connection is successfully established. The other is rejected with an RST (reset) flag.

Congestion Control Techniques

1. **Open Loop Congestion Control:** Prevents congestion by applying policies before congestion occurs. Examples:
 - **Retransmission Policy:** Ensures lost or corrupted packets are retransmitted without unnecessarily increasing congestion.
 - **Window Policy:** Adjusts the window size based on the number of packets successfully received to avoid congestion.
 - **Discarding Policy:** Routers may discard less important packets to avoid congestion while maintaining quality (e.g., in real-time audio transmission).
2. **Closed Loop Congestion Control:** Alleviates congestion once it has occurred through techniques such as:
 - **Backpressure:** Stops the upstream nodes from sending data if a node becomes congested.
 - **Choke Packet Technique:** A router sends a choke packet to the source to reduce traffic when congestion occurs.
 - **Explicit Signaling:** A congested node explicitly sends a message to inform the source of the congestion.

Approaches to Congestion Control

1. **Network Design Matching:** Ensure the network design is well-suited for the traffic it carries, avoiding congestion by properly sizing resources like bandwidth.
2. **Dynamic Resource Addition (Provisioning):** Adding network resources (e.g., routers, links) during congestion to accommodate increasing traffic.
3. **Traffic-Aware Routing:** Routers can adjust their behavior based on network traffic patterns to prevent congestion during peak usage times.
4. **Admission Control:** New connections are denied if they would contribute to network congestion.
5. **Load Shedding:** Routers discard some packets to reduce congestion, ensuring the network continues to function without collapsing.

The Function of TCP/IP Protocol

The main task of **TCP/IP** is to ensure reliable and accurate data transfer between devices. It breaks data into **packets** at the sender's side, which are reassembled at the receiver's side to maintain data accuracy. This segmentation and reassembly process ensures that large data sets can be efficiently transmitted across the network without loss of integrity.

The **TCP/IP model** divides the data into four layers:

1. **Application Layer:** Data is generated and processed.
2. **Transport Layer:** Data is divided into packets for transmission.
3. **Network Layer:** Responsible for routing packets through the network.
4. **Data Link and Physical Layers:** Packets are transmitted through physical media to the destination.

Congestion Control techniques

